

PIMPRI CHINCHWAD EDUCATION TRUST'S

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

NBA Accredited | NAAC Accredited with 'A' Grade | An Autonomous Institute
AICTE Approved | Permanently Affiliated to SPPU, Pune

Department of Information Technology

Academic Year: 2026-2027

Assignment No. 7: University Timetable Scheduling (Graph Colouring)

Name	:	**Om Rajendra Bharambe**
PRN	:	124B1F001
Branch	:	Information Technology (IT)
Semester	:	V
Batch	:	A1
Course Code	:	BIT25PC04
Course Name	:	**Design & Analysis of Algorithms Laboratory**
Guided By	:	Dr. Rajesh N. Phursule
GitHub Repository	:	om29dev/DAA_Lab_Submission
Assignment Link	:	Assignment 7 on GitHub

Assignment No. 7: University Timetable Scheduling

Scenario: University Timetable Scheduling

A university is facing challenges in scheduling exam timetables due to overlapping student enrolments in multiple courses. To prevent clashes, the university needs to assign exam slots efficiently, ensuring that no two exams taken by the same student are scheduled at the same time.

To solve this, the university decides to model the problem as a Graph Colouring Problem, where:

- Each course is represented as a vertex.
- An edge exists between two vertices if a student is enrolled in both courses.
- Each vertex (course) must be assigned a colour (time slot) such that no two adjacent vertices share the same colour (no two exams with common students are scheduled in the same slot).

As a scheduling system developer, you must:

1. Model the problem as a graph and implement a graph colouring algorithm (e.g., Greedy Colouring or Backtracking).

2. Minimize the number of colours (exam slots) needed while ensuring conflict-free scheduling.

3. Handle large datasets with thousands of courses and students, optimizing performance.

4. Compare the efficiency of Greedy Colouring, DSATUR, and Welsh-Powell algorithms for better scheduling.

Extend the solution to include room allocation constraints where exams in the same slot should fit within available classrooms.

Aim:

To design an efficient exam timetable scheduling system using Graph Colouring, ensuring that no two exams with common students are held at the same time while minimizing the total number of exam slots.

Course Outcomes:

- Apply Backtracking strategies to solve various problems.
- Classify P, NP, NP-complete, NP-Hard problems.

Course Objectives:

- To know the basics of computational complexity of various algorithms.

- To select appropriate algorithm design strategies to solve real-world problems.

Theory:

1. Graph Theory:

Graph Theory is a mathematical concept used to model pairwise relationships between objects. A graph G(V, E) consists of:

- **Vertices (V):** Represent entities such as courses.
- **Edges (E):** Represent relationships between entities, such as shared students between courses.

Graphs can be:

- **Directed or Undirected:** Depending on whether relationships have direction.
- **Weighted or Unweighted:** Depending on whether edges carry weights like distance or cost.

In this scheduling problem, the graph is undirected and unweighted, as the relationship (conflict) between two courses is mutual and has no weight.

2. Graph Colouring:

Graph Colouring is the process of assigning colors to vertices of a graph so that no two adjacent vertices have the same colour.

In the context of timetable scheduling:

- Each course is a vertex.
- An edge connects two vertices if at least one student is enrolled in both courses.
- Each colour represents a distinct exam time slot.

The goal is to minimize the total number of colours (time slots) used. The minimum number of colours required to colour a graph without conflicts is called its Chromatic Number.

3. Conflict Graph (Problem Modeling):

To prevent overlapping exams:

1. Courses with common students are connected by edges.
2. If two courses share an edge, they cannot have the same exam slot.

Thus, the timetable scheduling becomes equivalent to a Graph Coloring Problem, where no two connected nodes (conflicting courses) can share the same color (exam time). This approach efficiently models real-world exam conflicts and ensures no student faces two exams at the same time.

4. Graph Colouring Algorithms:

a. Greedy Coloring Algorithm:

- Colors one vertex at a time using the smallest possible color not used by its adjacent vertices.
- Although simple, it does not always produce the minimum number of colors.
- Time Complexity: O(V^2)
- Advantages: Easy to implement and works well for large graphs.

b. DSATUR (Degree of Saturation) Algorithm:

- Chooses the next vertex to color based on the number of different colors used by its adjacent vertices.
- Produces near-optimal solutions and handles complex conflict graphs effectively.
- Time Complexity: O(V^2)

c. Welsh-Powell Algorithm:

- Sorts vertices in descending order of their degrees (most connected first).
- Then applies the greedy method to color vertices.
- Performs better than simple greedy in most cases.
- Time Complexity: O(V^2)

5. Room Allocation Constraint:

After time slots (colours) are assigned, each exam in the same slot must be assigned to available classrooms. Room allocation is done based on:

3. **Room capacity:** Each exam must fit within the capacity of the assigned room.
4. **Availability:** A room can only host one exam per time slot.
5. **Optimization:** Allocate the smallest available room that fits all students to maximize space utilization.

This ensures effective use of university resources and avoids scheduling conflicts both in time and space.

6. Optimization for Large Datasets:

For large universities with thousands of students and courses:

- Use Adjacency Lists (instead of matrices) to save memory.
- Use Hash Maps or Dictionaries to store student-course relationships.
- Apply sorting and efficient traversal to reduce complexity.
- Implement parallelization when building the conflict graph for faster processing.

Pseudo code:

```
Input:
  C = list of all courses
  S = list of all students with their enrolled courses
  R = list of available rooms with capacities
Output:
  Timetable showing (Course -> Time Slot -> Room)

Begin
  // Step 1: Build Conflict Graph
  Create an empty graph G with vertices as courses in C
  For each student s in S do
    courses_list = courses enrolled by s
    For each pair (c1, c2) in courses_list do
      Add an edge between c1 and c2 in G
    EndFor
  EndFor

  // Step 2: Initialize Colors
  For each course c in C do
    color[c] <- -1 // no color assigned initially
  EndFor

  // Step 3: Apply Graph Coloring (Greedy Method)
  Sort all courses in descending order of their degree (number of edges)
  color[first_course] <- 0
  For each course u in sorted list (excluding first_course) do
    Create available_colors[] <- true for all colors
    For each adjacent course v of u do
      If color[v] != -1 then
        available_colors[color[v]] <- false
      EndIf
    EndFor
    Assign the smallest color i where available_colors[i] = true
    color[u] <- i
  EndFor

  // Step 4: Room Allocation for Each Time Slot
  For each time_slot t in set of used colors do
    exams_in_slot <- all courses having color = t
    Sort exams_in_slot by number_of_students (descending)
    Sort rooms R by capacity (descending)
    For each course e in exams_in_slot do
      For each room r in R do
        If capacity(r) >= students_in(e) AND r is available for slot t then
```

```
          Assign room r to course e
          Mark r as occupied for slot t
          Break
        EndIf
      EndFor
    EndFor
  EndFor

  // Step 5: Display Final Timetable
  For each course c in C do
    Print "Course:", c, "-> Time Slot:", color[c], "-> Room:",
assigned_room[c]
  EndFor
End
```

Example Input:

Student	Enrolled Courses
S1	C1, C2
S2	C2, C3
S3	C1, C3
S4	C4

Room	Capacity
R1	100
R2	80
R3	50

Expected Output:

Course	Time Slot	Room
C1	Slot 1	R1
C2	Slot 2	R2

C3	Slot 3	R3
C4	Slot 1	R2

Explanation of Output:

1. Courses C1, C2, and C3 conflict with each other (same students), hence different slots.
2. C4 has no conflict, so it shares Slot 1 with C1.
3. Each course is assigned a room according to its capacity and availability.

Conclusion:

The university timetable scheduling problem is efficiently solved using the Graph Colouring technique, where each course is represented as a vertex and conflicts are modelled as edges. By applying colouring algorithms such as the Greedy method, the system ensures that no two exams with common students are scheduled in the same time slot. Extending this approach with room allocation based on capacity further optimizes resource usage. Hence, the proposed model provides a conflict-free, scalable, and practical solution for exam scheduling in universities.

Code and Output:

Repository: https://github.com/om29dev/DAA_Lab_Submission

Assignment 7 Folder: https://github.com/om29dev/DAA_Lab_Submission/tree/main/Assignment 7

7.1 C++ Implementation Source Code (exam_timetable_graph_colouring.cpp):

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <map>

using namespace std;

struct Room {
    string name;
    int capacity;
};

int main() {
    int numCourses;
    cout << "Enter number of courses: ";
```

```
    if (!(cin >> numCourses)) return 0;

    vector<string> courses(numCourses);
    map<string, int> courseIndex;
    cout << "Enter course codes:\n";
    for (int i = 0; i < numCourses; i++) {
        cin >> courses[i];
        courseIndex[courses[i]] = i;
    }

    int numStudents;
    cout << "Enter number of students: ";
    cin >> numStudents;

    vector<int> studentCount(numCourses, 0);
    vector<vector<int>> adj(numCourses, vector<int>(numCourses, 0));

    cout << "Enter student enrollments (student_id count course1 course2
...):\n";
    for (int i = 0; i < numStudents; i++) {
        string sId;
        int count;
        cin >> sId >> count;
        vector<int> enrolled;
        for (int j = 0; j < count; j++) {
            string cCode;
            cin >> cCode;
            if (courseIndex.find(cCode) != courseIndex.end()) {
                int cIdx = courseIndex[cCode];
                studentCount[cIdx]++;
                enrolled.push_back(cIdx);
            }
        }
        for (size_t a = 0; a < enrolled.size(); a++) {
            for (size_t b = a + 1; b < enrolled.size(); b++) {
                int u = enrolled[a];
                int v = enrolled[b];
                adj[u][v] = 1;
                adj[v][u] = 1;
            }
        }
    }

    int numRooms;
    cout << "Enter number of available classrooms: ";
    cin >> numRooms;
```

```
vector<Room> rooms(numRooms);
cout << "Enter room name and capacity:\n";
for (int i = 0; i < numRooms; i++) {
    cin >> rooms[i].name >> rooms[i].capacity;
}

vector<int> degree(numCourses, 0);
for (int i = 0; i < numCourses; i++) {
    for (int j = 0; j < numCourses; j++) {
        if (adj[i][j]) degree[i]++;
    }
}

vector<int> order(numCourses);
for (int i = 0; i < numCourses; i++) order[i] = i;

sort(order.begin(), order.end(), [&](int a, int b) {
    return degree[a] > degree[b];
});

vector<int> color(numCourses, -1);
for (int idx = 0; idx < numCourses; idx++) {
    int u = order[idx];
    vector<bool> used(numCourses + 1, false);
    for (int v = 0; v < numCourses; v++) {
        if (adj[u][v] && color[v] != -1) {
            used[color[v]] = true;
        }
    }
    int assignedColor = 0;
    while (used[assignedColor]) assignedColor++;
    color[u] = assignedColor;
}

int maxColor = 0;
for (int i = 0; i < numCourses; i++) {
    maxColor = max(maxColor, color[i]);
}

cout << "\nExam Timetable Schedule:\n";
cout << "Total Time Slots Required: " << (maxColor + 1) << "\n\n";

vector<string> assignedRoom(numCourses, "Unassigned");
```

```
    for (int slot = 0; slot <= maxColor; slot++) {
        vector<int> slotCourses;
        for (int i = 0; i < numCourses; i++) {
            if (color[i] == slot) slotCourses.push_back(i);
        }

        sort(slotCourses.begin(), slotCourses.end(), [&](int a, int b) {
            return studentCount[a] > studentCount[b];
        });

        vector<bool> roomOccupied(numRooms, false);

        for (int cIdx : slotCourses) {
            int needed = studentCount[cIdx];
            int bestRoom = -1;
            int minCapacityFit = 1e9;

            for (int r = 0; r < numRooms; r++) {
                if (!roomOccupied[r] && rooms[r].capacity >= needed) {
                    if (rooms[r].capacity < minCapacityFit) {
                        minCapacityFit = rooms[r].capacity;
                        bestRoom = r;
                    }
                }
            }

            if (bestRoom != -1) {
                assignedRoom[cIdx] = rooms[bestRoom].name;
                roomOccupied[bestRoom] = true;
            }
        }
    }

    for (int i = 0; i < numCourses; i++) {
        cout << "Course: " << courses[i]
             << " -> Time Slot: Slot " << (color[i] + 1)
             << " -> Room: " << assignedRoom[i]
             << " (Students: " << studentCount[i] << ")\n";
    }

    return 0;
}
```

7.2 Execution Output:

Compile and run exam_timetable_graph_colouring.cpp in the terminal environment:

```
om29dev@DESKTOP-MH6NVIC:~/learn$ vi exam_timetable_graph_colouring.cpp
om29dev@DESKTOP-MH6NVIC:~/learn$ g++ exam_timetable_graph_colouring.cpp -o exam_timetable_graph_colouring
om29dev@DESKTOP-MH6NVIC:~/learn$ ./exam_timetable_graph_colouring
Enter number of courses: 4
Enter course codes:
C1 C2 C3 C4
Enter number of students: 4
Enter student enrollments (student_id count course1 course2 ...):
S1 2 C1 C2
S2 2 C2 C3
S3 2 C1 C3
S4 1 C4
Enter number of available classrooms: 3
Enter room name and capacity:
R1 100
R2 80
R3 50

Exam Timetable Schedule:
Total Time Slots Required: 3

Course: C1 -> Time Slot: Slot 1 -> Room: R3 (Students: 2)
Course: C2 -> Time Slot: Slot 2 -> Room: R3 (Students: 2)
Course: C3 -> Time Slot: Slot 3 -> Room: R3 (Students: 2)
Course: C4 -> Time Slot: Slot 1 -> Room: R2 (Students: 1)
om29dev@DESKTOP-MH6NVIC:~/learn$
```

Figure 7.1: University Exam Timetable Scheduling Execution Output